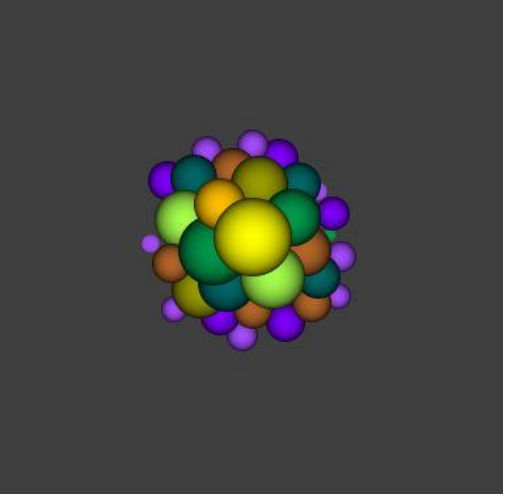
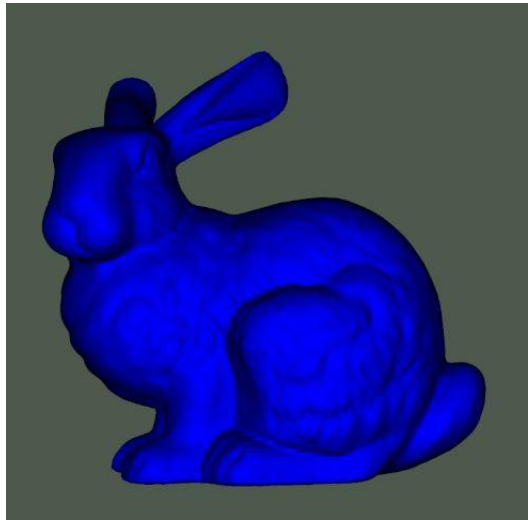


CS 440: Computer Graphics

Week 3

Lecture 5: Imaging & Graphics Pipeline



Acknowledgements: Cem Yuksel, University of Utah; Waqar Saleem, Habib University

Announcements and Recap

- Change in office hours this week
  - Wed: 10:30am – 11:30am
- Quiz next week

Recap: Imaging

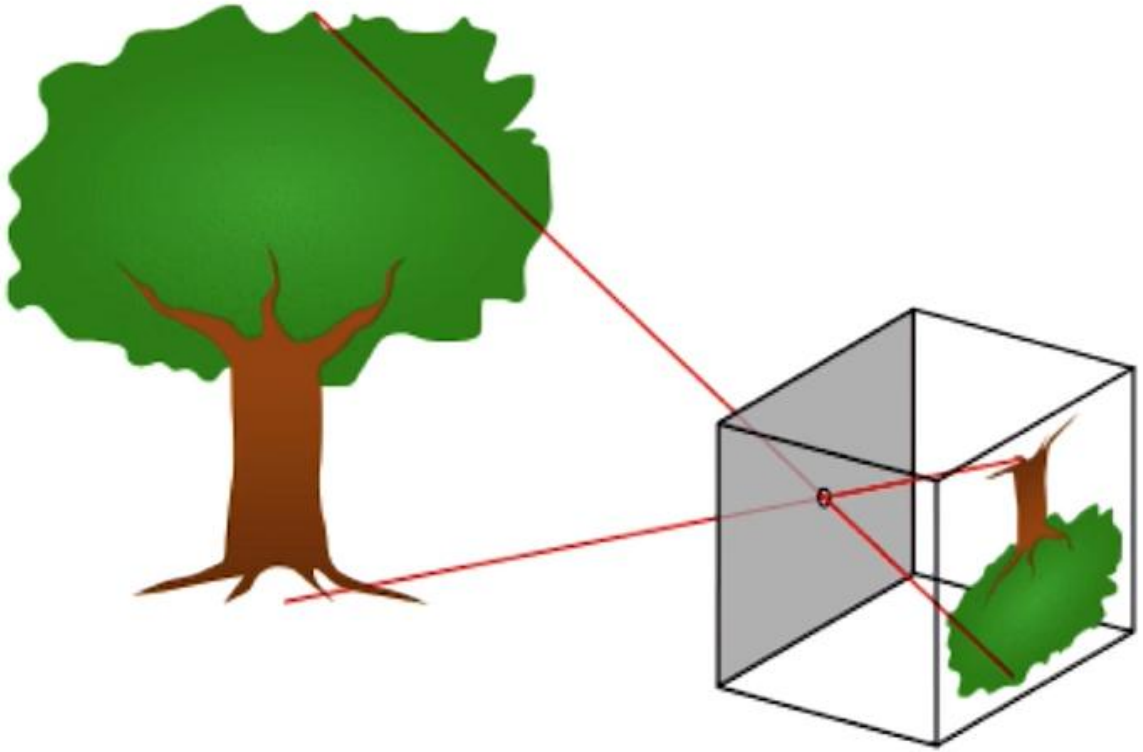
- Color
- Image Formation
- Imaging System: Pinhole Camera

Today: Imaging System, Graphics Pipeline & WebGL

- Imaging System: Synthetic Camera Model
- Rasterization based Graphics Pipeline
- Stages of Graphics Pipeline
- WebGL Context

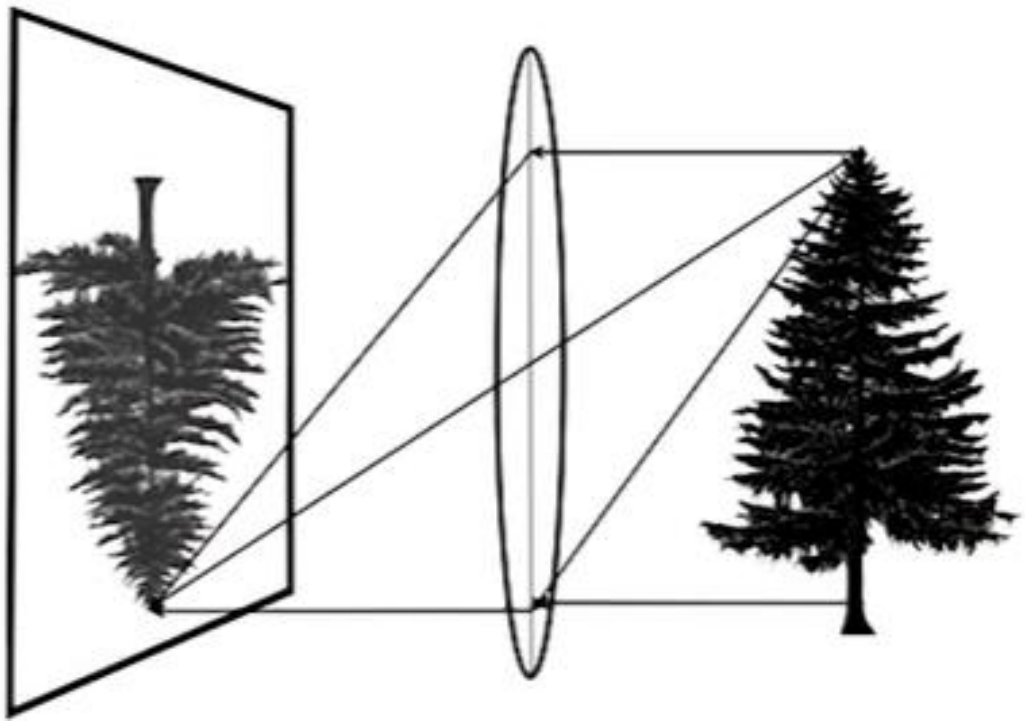
Recall: Pinhole Camera

- All light from the 3D scene goes through the pinhole
- A screen/sensor at the back of the box (opposite of pinhole) creates an inverted image
- A camera would “preserve” this image

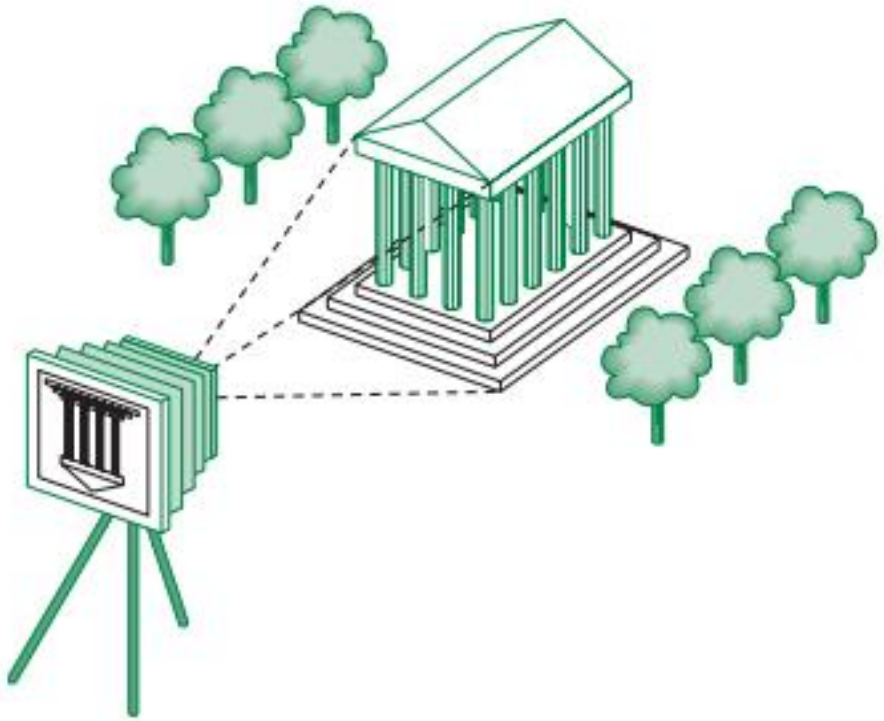


Replacing Pinhole with a Lens

- Lens have a larger aperture, so more light is collected
- Light rays converge at a point (similar to pinhole)
- No infinite depth of field



Imaging System: Synthetic Camera Model



Imaging System: Synthetic Camera Model

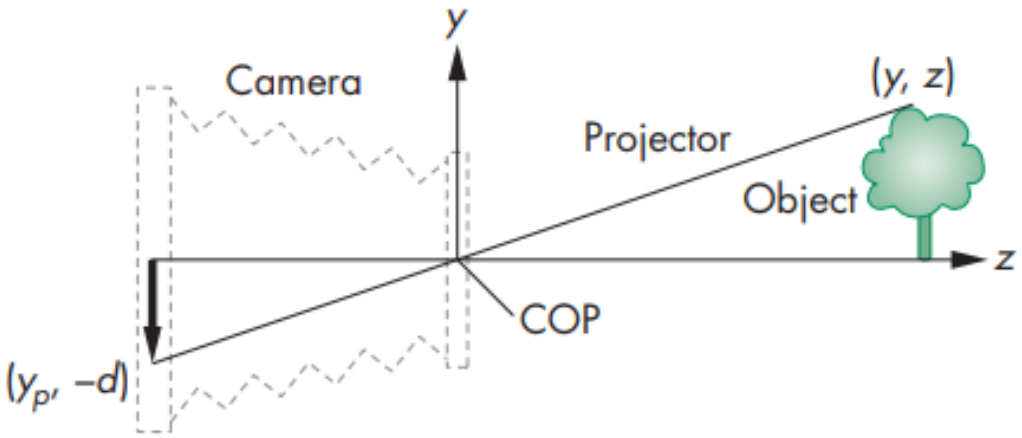


Image formed at back of camera

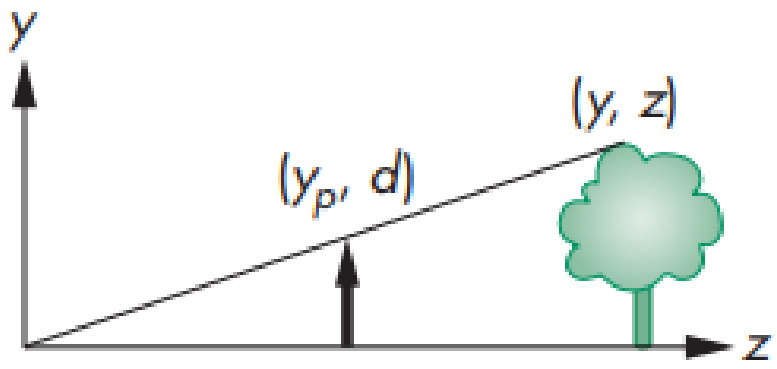
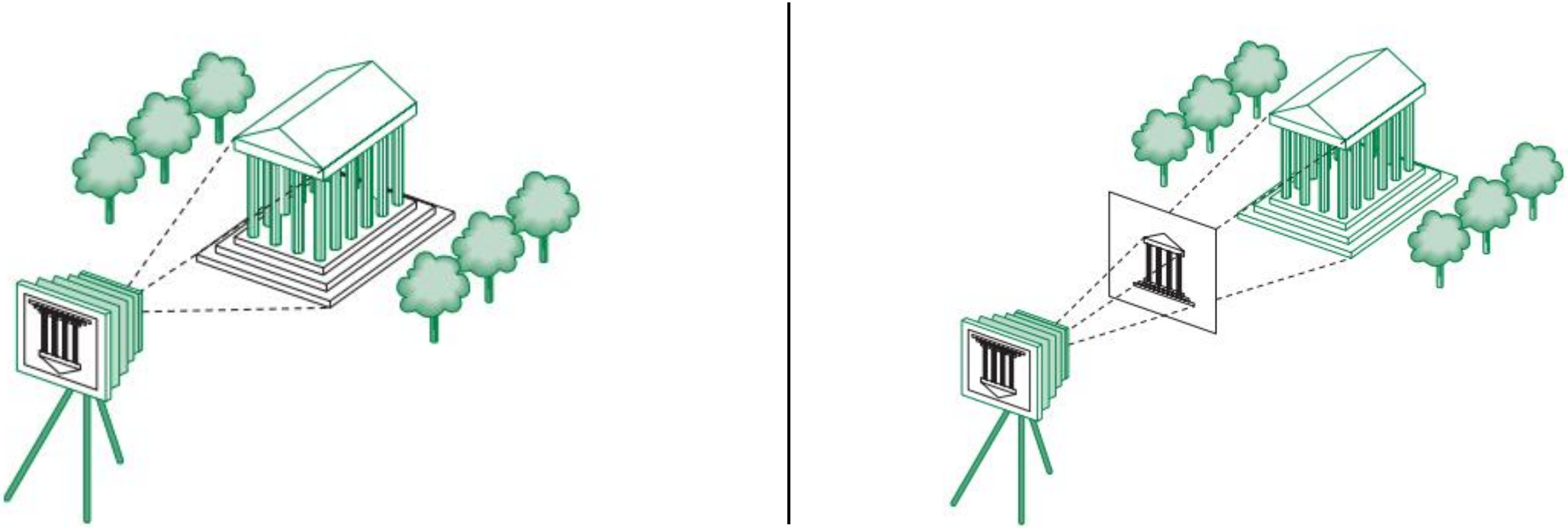
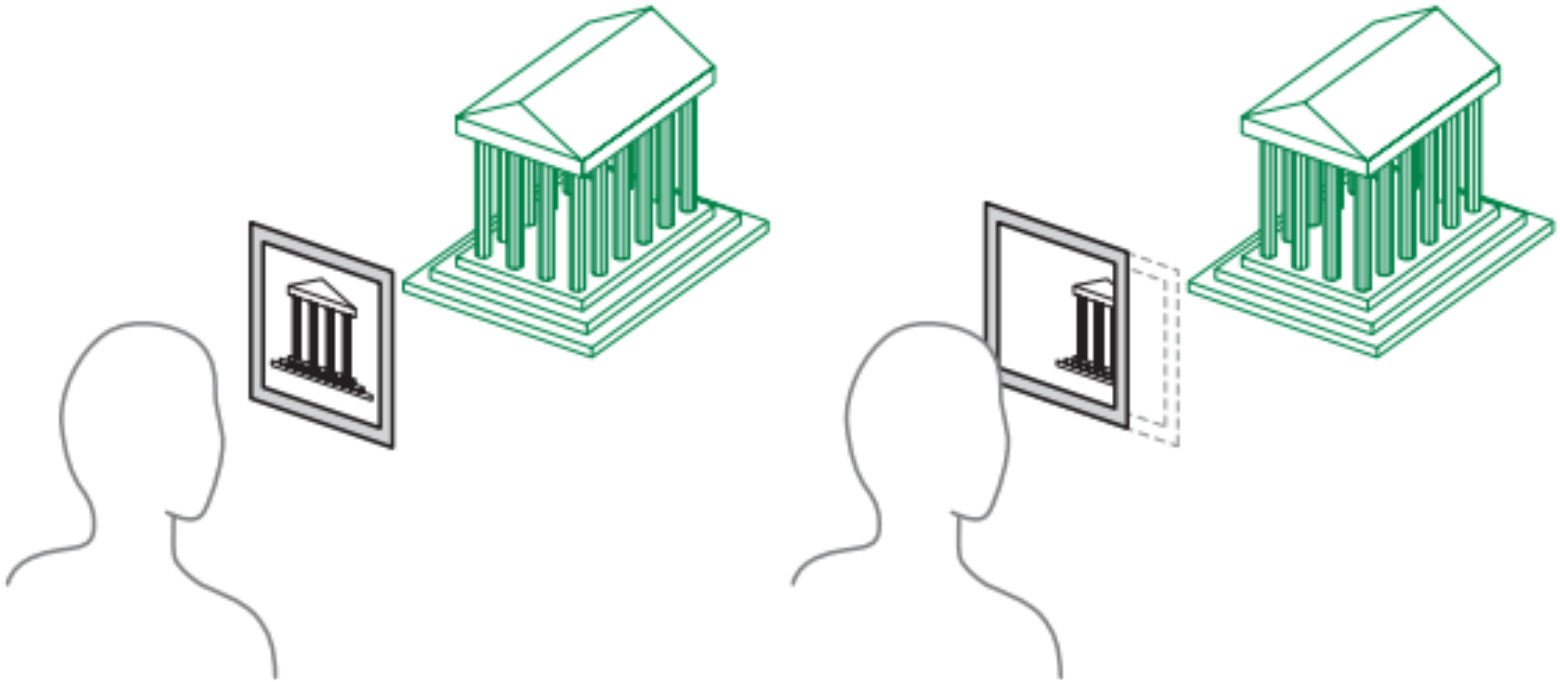


Image plane at the front of the camera

Imaging System: Synthetic Camera Model

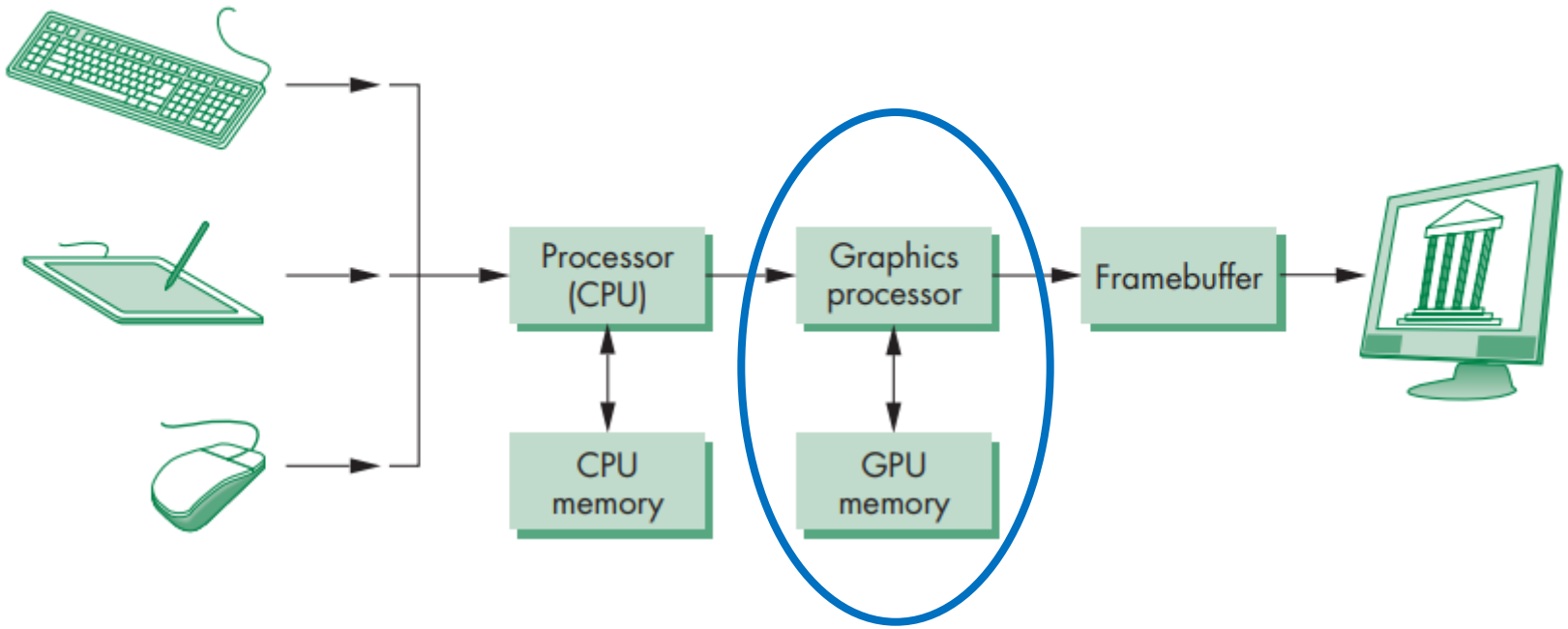


Imaging System: Synthetic Camera Model



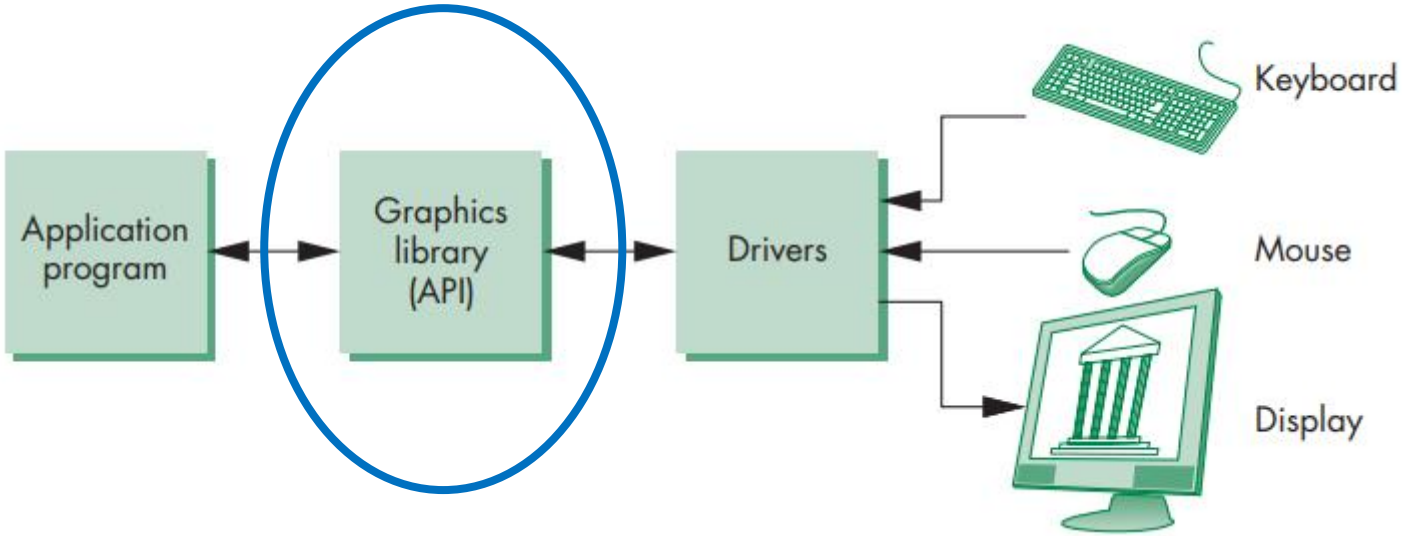
Recall: Systems View

- Intermediate steps between GPU and Framebuffer



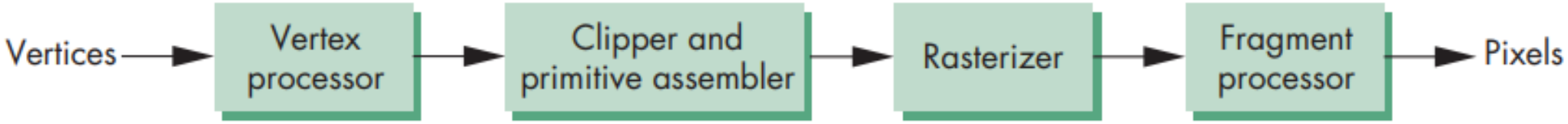
Recall: Graphics API

- Programmer's View of a graphics system
- Independent of I/O details



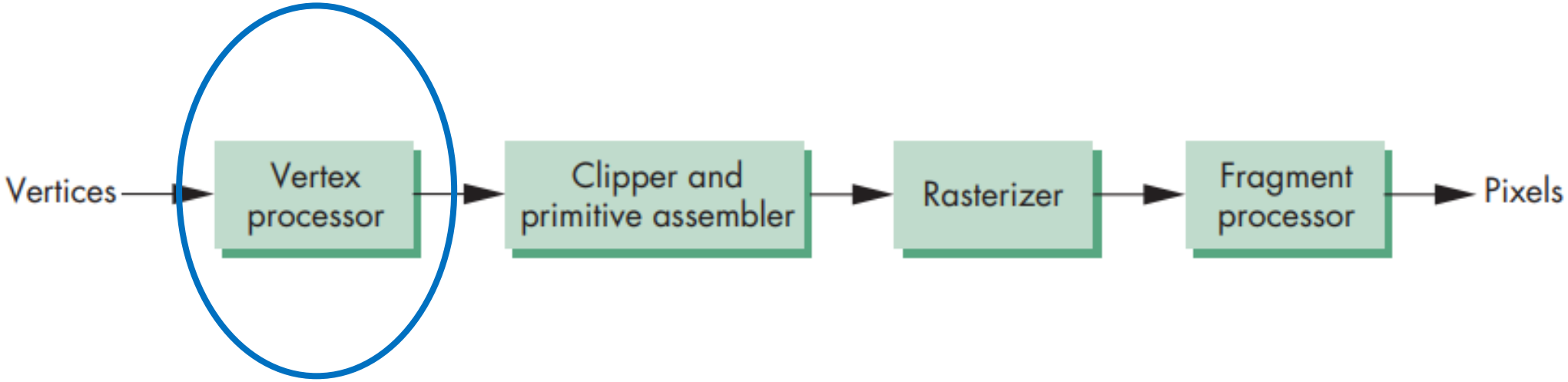
Graphics Pipeline

- Combination of hardware and software that implements the functionality of the API



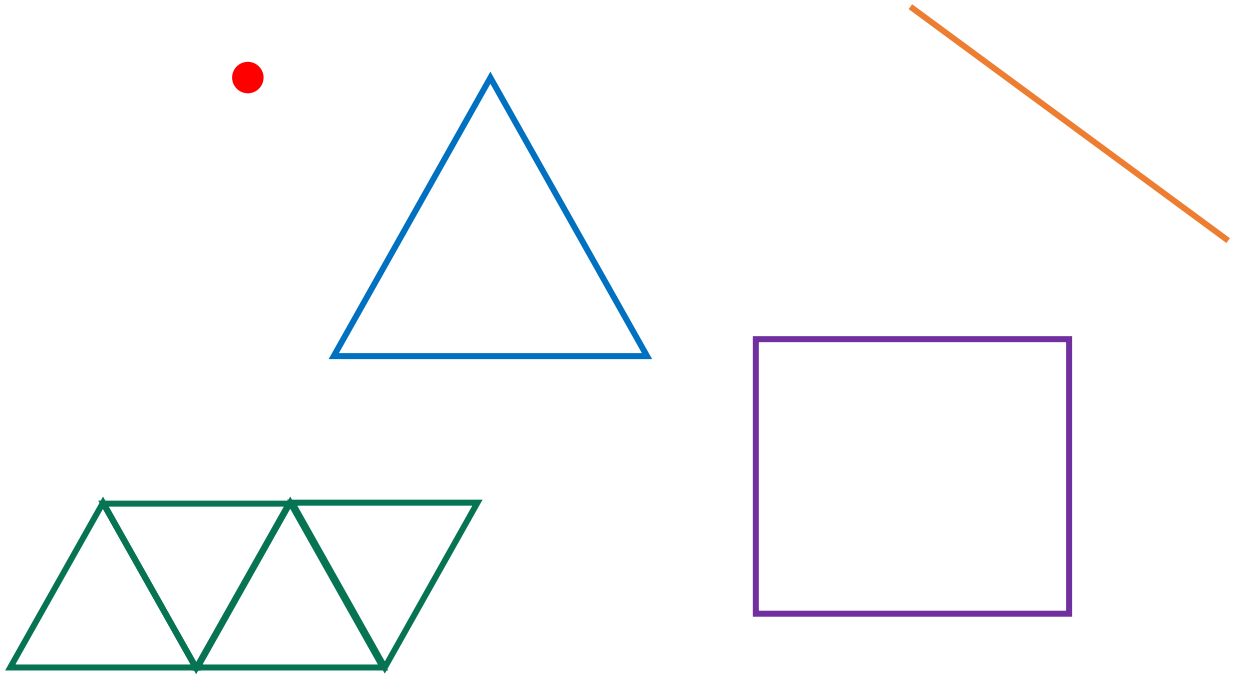
Graphics Pipeline: Vertex Processor

- Vertex Shader: Programmable
- Each vertex is processed independently
- Carries out “per-vertex” computations



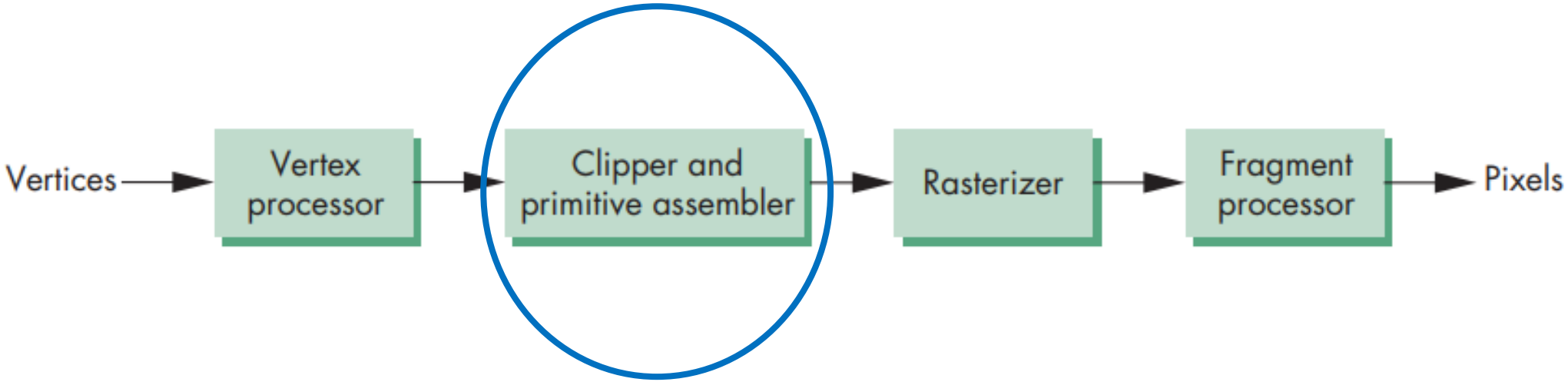
Primitives

- Simplest geometric shape that a graphical system can draw
  - Triangle
  - Point
  - Line
  - Quads
  - Triangle Strip



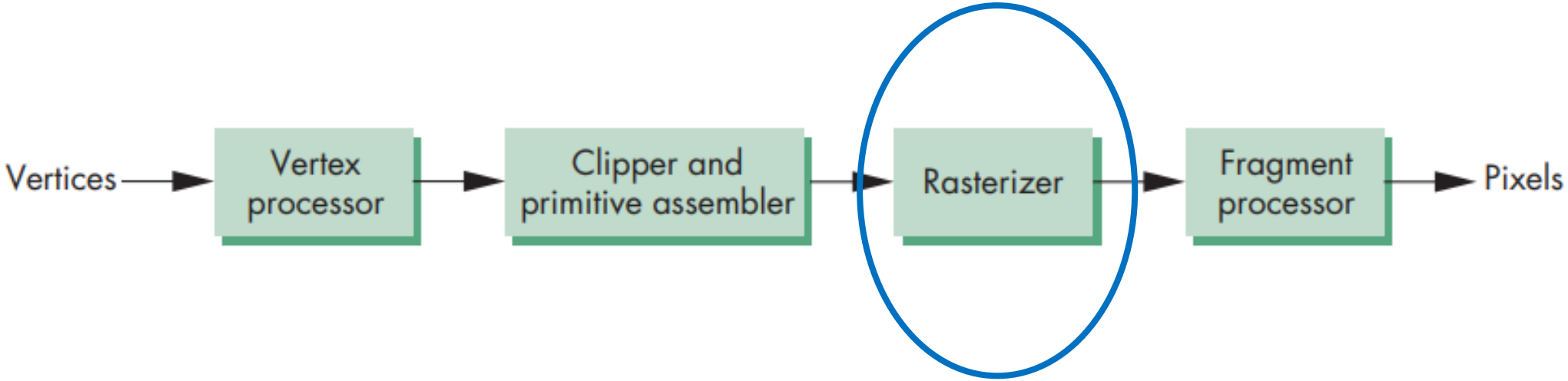
Graphics Pipeline: Clipper and Primitive Assembly

- Assemble vertices to form **primitives**
- Clipping parts that are not visible



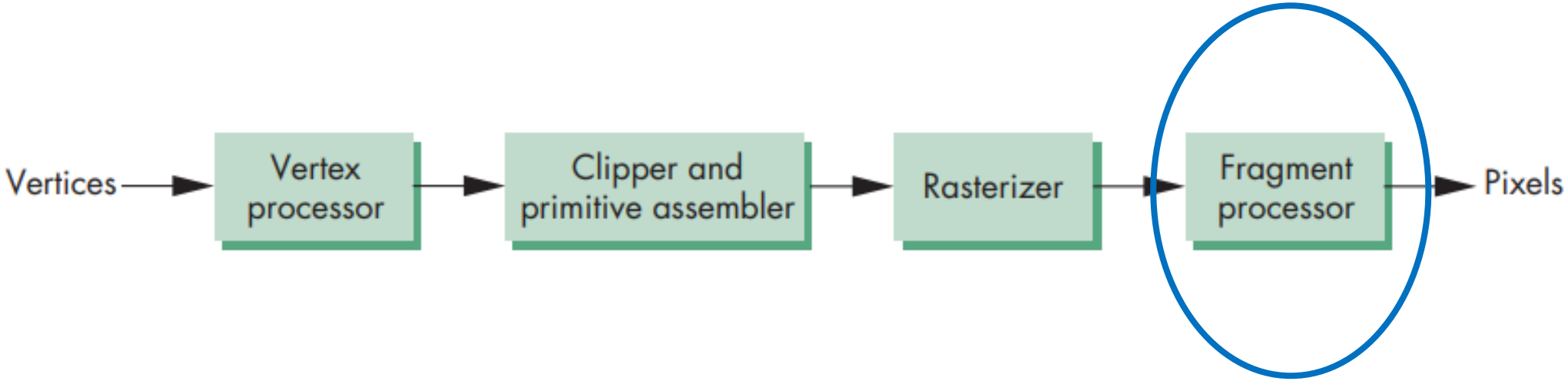
Graphics Pipeline: Rasterizer

- Vector data to raster image data
- Transformed vertices converted to **fragments**

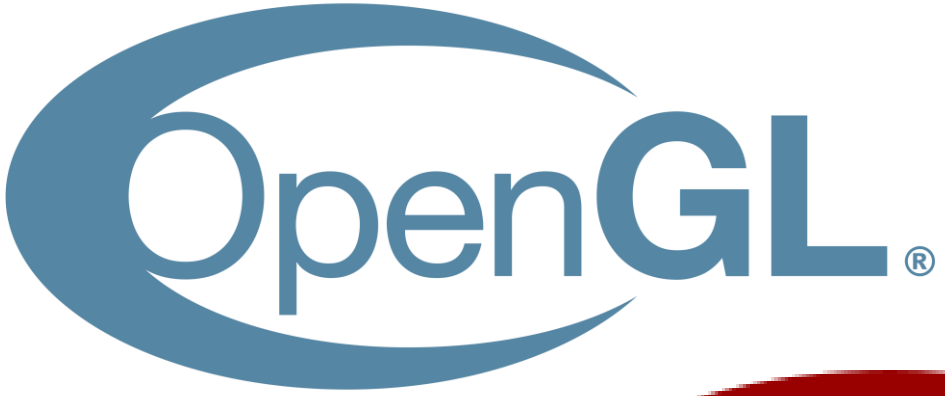


Graphics Pipeline: Fragment Processor

- Fragment Shader: Programmable
- Each fragment is processed independently
- Carries out “per-fragment” computations



WebGL Origins



Anatomy of a WebGL Program

- Initialize the system
  - Connect with HTML Page
  - Specify shaders
- Compute the scene
  - Application logic
- Send scene data to GPU
  - Reserve GPU memory
  - Copy (vertex) data to GPU buffers
- Render scene
  - Trigger the pipeline

Anatomy of a WebGL Program

- Initialize the system
  - Connect with HTML Page
  - Specify shaders
- Compute the scene
  - Application logic
- Send scene data to GPU
  - Reserve GPU memory
  - Copy (vertex) data to GPU buffers
- Render scene
  - Trigger the pipeline